

MANUAL DE PROGRAMADOR

VENTUM

INTEGRANTES:

Juan José Miranda Solís

Jacobo Villegas Figueroa

Duván Blandón Obando

PROYECTO DE VIDA

OCTUBRE DEL 2025

COLEGIO DE LA UNIVERSIDAD PONTIFICIA BOLIVARIANA

ÍNDICE

1. Introducción
2. Librerías utilizadas
3. Estructura general del sistema
4. update_admin_password.py
5. sesión.py
6. Ventanas principales
 - a. Ventana Login
 - b. Ventana Principal (Dashboard)
 - c. Ventana Ventas
 - d. Ventana Nueva Venta
 - e. Ventana Clientes
 - f. Ventana Inventario
 - g. Ventana Reportes
7. Ventana inventario
8. Ventana facturación
9. Ventana bd
10. Models
 - a. Ventas.py
 - b. usuario.py
 - c. reportes.py
 - d. productos.py

e. clientes.py

f. créditos.py

11. lógica_iu

a. Registro_usuarios.py

b. Registr_cliente.py

c. registrar_usuario_window.py

d. registrar_abono.py

e. clientes.py

f. admin_product_create.py

g. admin_client_list.py

h. admin_client_edit.py

1. Introducción

El presente manual tiene como objetivo explicar el funcionamiento del código del sistema de ventas, ventum, desarrollado en Python utilizando la librería PyQt6.

Este sistema cuenta con una interfaz gráfica compuesta por diferentes ventanas que permiten gestionar usuarios, clientes, productos, ventas e informes.

A lo largo del documento se explicará la estructura del proyecto, el propósito de cada clase, y el flujo lógico que sigue la aplicación.

2. Librerías utilizadas

El proyecto utiliza varias librerías esenciales para el desarrollo de su interfaz gráfica y su funcionalidad interna:

- PyQt6: Permite crear interfaces gráficas con ventanas, botones, formularios y tablas.
- matplotlib: Se usa para generar gráficos estadísticos, como ventas mensuales o ventas por categoría.
- io: Maneja flujos de datos binarios (como la conversión de gráficos en imágenes).
- models: Carpeta que contiene los módulos que gestionan la lógica de negocio (usuarios, clientes, ventas, productos, reportes).

```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

import sys

from PyQt6 import QtWidgets, uic

# Importaciones de los modelos
from models.usuario import validar_usuario
from models.ventas import listar_ventas, registrar_venta, listar_tipos_pago
from models.clientes import listar_clientes, agregar_cliente
from models.productos import listar_productos
from models.reportes import productos_mas_vendidos, clientes_frecuentes, saldos_pendientes, liquidaciones_ventas
```

3. Estructura general del sistema

El sistema está organizado en distintas clases, cada una representando una ventana o módulo del programa.

Las principales clases son:

- LoginWindow: Gestiona el inicio de sesión del usuario.

```
class LoginWindow(QtWidgets.QDialog):
```

- MainWindow: Es la ventana principal o panel de control.

```
class MainWindow(QtWidgets.QMainWindow):
```

- VentasWindow y NuevaVentaWindow: Permiten gestionar las ventas y registrar nuevas.

```
class VentasWindow(QtWidgets.QMainWindow):
```

```
class NuevaVentaWindow(QtWidgets.QMainWindow):
```

- ClientesWindow y RegistroClienteWindow: Manejan el listado y registro de clientes.

```
class ClientesWindow(QtWidgets.QMainWindow):
```

```
class RegistroClienteWindow(QtWidgets.QMainWindow):
```

- InventarioWindow: Muestra los productos disponibles.

```
class InventarioWindow(QtWidgets.QMainWindow):
```

- ReportesWindow: Genera reportes y estadísticas.

```
class ReportesWindow(QtWidgets.QMainWindow):
```

4. update_admin_password.py

Este módulo tiene como propósito **actualizar y cifrar la contraseña del usuario administrador** (admin) dentro de la base de datos del sistema *Ventum*.

Utiliza el algoritmo **bcrypt** para garantizar que las contraseñas no se almacenen en texto plano, reforzando así la **seguridad de las credenciales** en el sistema.

También realiza una **verificación previa del estado actual de la contraseña**, identificando si ya está cifrada o si requiere reparación.

Funcionalidades principales

1. Conexión a la base de datos.
2. Verificación del usuario administrador.
3. Cifrado de la nueva contraseña.
4. Actualización en la base de datos.
5. Mensajes de diagnóstico y éxito.

Configuración de parámetros del sistema

Se define la información necesaria para la conexión a la base de datos y las credenciales a actualizar.

Estos valores son configurables según el entorno del servidor.

```
# --- CONFIGURACIÓN ---  
DB_CONFIG = {  
    'host': 'localhost',  
    'user': 'root',  
    'password': '',  
    'database': 'ventum'  
}  
ADMIN_USERNAME = 'admin'  
NEW_PASSWORD = 'admin'  
# -----
```

Definición de la función principal

La función `update_admin_password()` contiene todo el flujo de operaciones:

1. Conexión a la base de datos.
2. Verificación del usuario administrador.
3. Diagnóstico de la contraseña actual.
4. Cifrado y actualización en la base de datos.
5. Cierre de conexión y confirmación visual.

```
def update_admin_password():
    """
    Este script se conecta a la base de datos y actualiza la contraseña del usuario 'admin'
    a una versión cifrada con bcrypt.
    """
    try:
        # 1. Conectar a la base de datos
        conn = mysql.connector.connect(**DB_CONFIG)
        cursor = conn.cursor(dictionary=True)

        # 2. Diagnóstico: Leer la contraseña actual
        cursor.execute("SELECT clave FROM usuario WHERE usuario = %s", (ADMIN_USERNAME,))
        user_data = cursor.fetchone()

        if not user_data:
            print(f"❌ ERROR DE DIAGNÓSTICO: No se encontró al usuario '{ADMIN_USERNAME}' en la base de datos.")
            return

        current_password = user_data['clave']
        print(f"🔍 Diagnóstico: La contraseña actual para '{ADMIN_USERNAME}' es: '{current_password}'")
        if not current_password.startswith('$2b$'):
            print("⚠️ La contraseña actual NO parece estar cifrada. Procediendo a repararla.")
        else:
            print("✅ La contraseña actual ya parece estar cifrada.")

        # 3. Reparación: Cifrar y actualizar la nueva contraseña
        # Cifrar la nueva contraseña
        hashed_password = bcrypt.hashpw(NEW_PASSWORD.encode('utf-8'), bcrypt.gensalt())
        print(f"🔐 Generando nuevo hash para la contraseña '{NEW_PASSWORD}': {hashed_password.decode()}")

        # Actualizar la base de datos
        cursor.execute("UPDATE usuario SET clave = %s WHERE usuario = %s", (hashed_password, ADMIN_USERNAME))
        conn.commit()

        print(f"\n✅ ¡Éxito! La contraseña del usuario '{ADMIN_USERNAME}' ha sido actualizada y cifrada correctamente en la base de datos.")

    except Error as e:
        print(f"❌ Error al conectar o actualizar la base de datos: {e}")
```

Cierre de conexión y ejecución directa

Finalmente, se cierran las conexiones activas a la base de datos y se permite la ejecución independiente del script.

```
if __name__ == '__main__':
    update_admin_password()
```

5. session.py

El módulo **SessionManager** es una parte esencial del sistema *Ventum*, encargada de **mantener la sesión activa del usuario que ha iniciado sesión en la aplicación**.

Su objetivo principal es **garantizar que los datos del usuario actual sean accesibles**

desde cualquier parte del sistema, sin necesidad de establecer múltiples conexiones ni volver a autenticar cada módulo.

Definición de la clase principal

El código define la clase SessionManager, encargada de gestionar los datos de la sesión del usuario dentro de la aplicación.

```
class SessionManager:
```

Implementación del método __new__

El método especial __new__ es el encargado de **controlar la creación del objeto**.

Cada vez que se intenta crear una nueva instancia de SessionManager, este método verifica si ya existe una instancia previa; si no existe, la crea y define el atributo current_user como None.

```
def __new__(cls):  
    if cls._instance is None:  
        cls._instance = super(SessionManager, cls).__new__(cls)  
        cls._instance.current_user = None  
    return cls._instance
```

Creación de una instancia global

Para facilitar el acceso al gestor de sesión, se crea una **instancia global** llamada **session**, la cual puede ser importada desde cualquier otro módulo del proyecto.

De esta manera, cualquier componente del sistema puede leer o modificar el usuario actual de forma sencilla.

```
session = SessionManager()
```

6. Ventanas principales

a. Ventana Login

La clase `LoginWindow` controla la autenticación del usuario. Carga el archivo de interfaz `login.ui` y valida los datos ingresados mediante la función `validar_usuario()`.

Si el usuario y contraseña son correctos, se abre la ventana principal.

```
def __init__(self):
    super().__init__()
    # Asegúrate que el archivo 'login.ui' existe
    uic.loadUi("ui/login.ui", self)

    self.btn_inicio_sesion.clicked.connect(self.login)
    self.lbl_mensaje.setText("")
    # Conectar el botón de crear nuevo usuario
    if hasattr(self, 'btn_nuevo_usuario'):
        self.btn_nuevo_usuario.clicked.connect(self.abrir_registro_usuario)
    elif hasattr(self, 'btn_nuevo_cliente'):
        self.btn_nuevo_cliente.clicked.connect(self.abrir_registro_usuario)

def abrir_registro_usuario(self):
    # Abrir ventana de registro de usuario usando el UI correspondiente
    self.registro_usuario = QtWidgets.QDialog()
    uic.loadUi("ui/register_usuario_fixed.ui", self.registro_usuario)
    self.registro_usuario.show()

def login(self):
    user = self.txt_correo.text()
    password = self.txt_clave.text()

    usuario = validar_usuario(user, password)
    if usuario:
        self.main = MainWindow()
        self.main.show()
        self.close()
    else:
        self.lbl_mensaje.setText("✗ Usuario o contraseña incorrectos")
```

b. Ventana Principal (Dashboard)

La clase `MainWindow` es el centro del sistema. Permite navegar a otras secciones como ventas, clientes, inventario o reportes.

Además, muestra gráficos estadísticos de las ventas mensuales y por categoría utilizando

```
def abrir_ventas(self):
    self.ventas = VentasWindow()
    self.ventas.show()

def abrir_clientes(self):
    self.clientes = ClientesWindow()
    self.clientes.show()

def abrir_inventario(self):
    self.inventario = InventarioWindow()
    self.inventario.show()

def abrir_reportes(self):
    self.reportes = ReportesWindow()
    self.reportes.show()

def abrir_nueva_venta(self):
    self.nueva_venta = NuevaVentaWindow()
    self.nueva_venta.show()
```

```
# Gráfica de ventas mensuales (barras)
self.graficoMensualWidget = pg.PlotWidget(parent=self.frameGraficos)
self.graficoMensualWidget.setTitle("Ventas Mensuales")
self.graficoMensualWidget.showGrid(x=True, y=True)
self.graficoMensualWidget.setLabel('left', 'Total Ventas')
self.graficoMensualWidget.setLabel('bottom', 'Mes')
```

c. Ventana Ventas

La clase `VentasWindow` carga una lista de todas las ventas registradas.

Cada venta muestra su ID, cliente, producto, fecha y total.

Desde esta ventana también se puede abrir el formulario de nueva venta.

```
def cargar_ventas(self):
    ventas = listar_ventas()
    self.tblVentas.setRowCount(0)

    for row in ventas:
        rowPosition = self.tblVentas.rowCount()
        self.tblVentas.insertRow(rowPosition)

        # ID Venta (columna 0)
        self.tblVentas.setItem(rowPosition, 0, QTableWidgetItem(str(row["id_venta"])))

        # Cliente (columna 1)
        self.tblVentas.setItem(rowPosition, 1, QTableWidgetItem(f'{row["nombre_cliente"]} {row["apellido_cliente"]}'))

        # Producto (columna 2) -> Ahora usa el nombre del producto
        self.tblVentas.setItem(rowPosition, 2, QTableWidgetItem(row["nombre_producto"]))

        # Fecha (columna 3)
        self.tblVentas.setItem(rowPosition, 3, QTableWidgetItem(str(row["fecha"])))

        # Total (columna 4)
        self.tblVentas.setItem(rowPosition, 4, QTableWidgetItem(str(row["total"])))

def abrir_nueva_venta(self):
    self.nueva = NuevaVentanaWindow()
    self.nueva.show()
```

d. Ventana Nueva Venta

`NuevaVentanaWindow` permite registrar una venta nueva.

Carga dinámicamente los clientes, productos y tipos de pago desde la base de datos, y valida los datos ingresados antes de registrar la transacción.

```
def cargar_datos(self):  
    # Cargar clientes  
    clientes = listar_clientes()  
    self.cmbCliente.clear()  
    for c in clientes:  
        self.cmbCliente.addItem(f'{c["nombre"]} {c["apellido"]}', c["id_cliente"])  
  
    # Cargar productos  
    productos = listar_productos()  
    self.cmbProducto.clear()  
    for p in productos:  
        self.cmbProducto.addItem(p["nombre"], p["id_producto"])  
  
    # Cargar tipos de pago  
    tipos_pago = listar_tipos_pago()  
    self.cmbTipoPago.clear()  
    for tp in tipos_pago:  
        self.cmbTipoPago.addItem(tp["nombre"], tp["id_tipo_pago"])
```

e. Ventana Clientes

La clase `ClientesWindow` lista todos los clientes almacenados en la base de datos.

Permite registrar nuevos clientes mediante la ventana `RegistroClienteWindow`.

```
def cargar_clientes(self):
    clientes = listar_clientes()
    self.tblClientes.setRowCount(0)
    for row in clientes:
        rowPosition = self.tblClientes.rowCount()
        self.tblClientes.insertRow(rowPosition)
        self.tblClientes.setItem(rowPosition, 0, QTableWidgetItem(str(row["id_cliente"])))
        self.tblClientes.setItem(rowPosition, 1, QTableWidgetItem(f'{row["nombre"]} {row["apellido"]}'))
        self.tblClientes.setItem(rowPosition, 2, QTableWidgetItem(row["correo"]))
        self.tblClientes.setItem(rowPosition, 3, QTableWidgetItem(row["ciudad"]))

def abrir_nuevo_cliente(self):
    self.nueva = RegistroClienteWindow()
    self.nueva.show()
```

e. Ventana Inventario

`InventarioWindow` muestra el listado de productos registrados, con su ID, nombre, precio y cantidad disponible en stock.

```
def cargar_inventario(self):
    productos = listar_productos()
    self.tablaInventario.setRowCount(0)
    for row in productos:
        rowPosition = self.tablaInventario.rowCount()
        self.tablaInventario.insertRow(rowPosition)
        self.tablaInventario.setItem(rowPosition, 0, QTableWidgetItem(str(row["id_producto"])))
        self.tablaInventario.setItem(rowPosition, 1, QTableWidgetItem(row["nombre"]))
        self.tablaInventario.setItem(rowPosition, 2, QTableWidgetItem(str(row["precio"])))
        self.tablaInventario.setItem(rowPosition, 3, QTableWidgetItem(str(row["stock"])))
```

f. Ventana Reportes

Esta clase genera diferentes reportes, como productos más vendidos, clientes frecuentes, saldos pendientes y liquidaciones.

Cada reporte se presenta en una tabla con información detallada.

```
def __init__(self):
    super().__init__()
    uic.loadUi("ui/reportes.ui", self)

    self.btnProductosMasVendidos.clicked.connect(self.cargar_productos_mas_vendidos)
    self.btnClientesFrecuentes.clicked.connect(self.cargar_clientes_frecuentes)
    self.btnSaldosPendientes.clicked.connect(self.cargar_saldos_pendientes)
    self.btnLiquidacionesVentas.clicked.connect(self.cargar_liquidaciones)

def cargar_productos_mas_vendidos(self):
    self.tblReportes.setRowCount(0)
    self.tblReportes.setColumnCount(2)
    self.tblReportes.setHorizontalHeaderLabels(["Producto", "Total Vendido"])

    mas_vendidos = productos_mas_vendidos()
    self.tblReportes.setRowCount(len(mas_vendidos))
    for rowPosition, row in enumerate(mas_vendidos):
        self.tblReportes.setItem(rowPosition, 0, QTableWidgetItem(row["nombre"]))
        self.tblReportes.setItem(rowPosition, 1, QTableWidgetItem(str(row["total_vendido"])))

def cargar_clientes_frecuentes(self):
    self.tblReportes.setRowCount(0)
    self.tblReportes.setColumnCount(2)
    self.tblReportes.setHorizontalHeaderLabels(["Cliente", "Compras Realizadas"])

    frecuentes = clientes_frecuentes()
    self.tblReportes.setRowCount(len(frecuentes))
    for rowPosition, row in enumerate(frecuentes):
        self.tblReportes.setItem(rowPosition, 0, QTableWidgetItem(f'{row["nombre"]} {row["apellido"]}')
        self.tblReportes.setItem(rowPosition, 1, QTableWidgetItem(str(row["compras"])))
```

7. Ventana de inventario

Este módulo define la **ventana de inventario del sistema**, encargada de mostrar los productos almacenados en la base de datos. Permite visualizar información como el **ID del producto, nombre, precio y stock disponible**.

```
import sys
from PyQt6 import QtWidgets, uic
from PyQt6.QtCore import Qt
from ui_py.inventario import Ui_InventarioWindow

# Importamos la clase de la base de datos
from db import Database
```

Estas importaciones permiten:

- Cargar y ejecutar interfaces gráficas (QtWidgets, uic, Qt).
- Acceder a la interfaz generada del archivo .ui.
- Conectarse a la base de datos mediante la clase Database.

La clase InventarioWindow representa la ventana donde se muestran los productos.

Su constructor (`__init__`) configura la interfaz, los botones de navegación y la conexión con la base de datos.

- `setupUi(self)`: carga todos los elementos visuales del archivo `inventario.ui`.
- Botones de navegación: permiten moverse entre las distintas secciones del sistema (inicio, ventas, etc.).
- `self.db = Database()`: crea una instancia de la clase `Database` para acceder a los datos.
- `self.cargar_productos()`: al abrir la ventana, se cargan automáticamente los productos en la tabla.

```
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
    self.setupUi(self)

    # Conectar los botones de la barra de navegación (ejemplo)
    self.btnInicio.clicked.connect(self.ir_a_inicio)
    self.btnVentas.clicked.connect(self.ir_a_ventas)
    # ... y así sucesivamente para los demás botones

    # Conectar la base de datos
    self.db = Database()

    # Llamar a la función para cargar los productos al iniciar la ventana
    self.cargar_productos()
```

Funciones de navegación

Estas funciones permiten moverse a otras ventanas del sistema:

```
def ir_a_inicio(self):  
    # Aquí iría el código para mostrar la ventana de inicio  
    print("Navegando a Inicio")  
  
def ir_a_ventas(self):  
    # Aquí iría el código para mostrar la ventana de ventas  
    print("Navegando a Ventas")
```

Función: cargar_productos()

Esta es la función principal del módulo.

Se encarga de obtener los datos desde la base de datos y mostrarlos en la tabla de la interfaz.

- **setRowCount(0)**: limpia la tabla antes de llenarla.
- **obtener_productos()**: función definida en el archivo db.py que consulta la base de datos.

Luego, si se encuentran productos, se llena la tabla:

Cada producto se coloca en una fila de la tabla, distribuyendo los datos en columnas:

1. **ID del producto**
2. **Nombre**
3. **Precio**
4. **Cantidad disponible (stock)**

Si no hay productos registrados, el sistema muestra un mensaje en la consola:

```
def cargar_productos(self):
    """
    Esta función se encargará de leer los datos de la tabla de productos
    y llenar la tabla de la interfaz.
    """
    # Limpiar la tabla antes de llenarla
    self.tablaInventario.setRowCount(0)

    # Obtener los productos de la base de datos
    # Esta función la definiremos en el archivo db.py
    productos = self.db.obtener_productos()

    if productos:
        # Llenar la tabla con los datos
        self.tablaInventario.setRowCount(len(productos))
        for fila, producto in enumerate(productos):
            # ID, Nombre, Precio, Stock
            id_producto = QTableWidgetItem(str(producto[0]))
            nombre = QTableWidgetItem(str(producto[3])) # [3] corresponde a la columna de nombre en tu tabla
            precio = QTableWidgetItem(str(producto[5])) # [5] corresponde a la columna de precio
            stock = QTableWidgetItem(str(producto[6])) # [6] corresponde a la columna de stock

            self.tablaInventario.setItem(fila, 0, id_producto)
            self.tablaInventario.setItem(fila, 1, nombre)
            self.tablaInventario.setItem(fila, 2, precio)
            self.tablaInventario.setItem(fila, 3, stock)
        else:
            print("No se encontraron productos.")
```

Bloque principal del programa

Este bloque permite ejecutar la ventana de forma independiente, útil para pruebas unitarias o desarrollo.

```
if __name__ == '__main__':
    app = QtWidgets.QApplication(sys.argv)
    window = InventarioWindow()
    window.show()
    sys.exit(app.exec())
```

8. Ventana facturación

Este módulo permite generar facturas en formato PDF a partir de la información de una venta registrada en el sistema.

Usa la librería ReportLab para crear documentos con formato profesional, incluyendo datos del cliente, productos vendidos y el total a pagar.

format_currency_pdf()

Formatea los números a estilo moneda.

Ejemplo → 25000 → \$25,000

Si el valor no es válido (por ejemplo, None o texto), lo devuelve como texto plano.

```
def format_currency_pdf(value):  
    """Formatea un número como una cadena de moneda para el PDF."""  
    try:  
        return f"${int(value):,}"  
    except (ValueError, TypeError):  
        return str(value)
```

generar_factura_pdf(datos_venta)

Generar una factura con los datos completos de la venta:

- Encabezado con número de factura y fecha.

- Información del cliente.
- Detalles de los productos.
- Totales con descuento y total a pagar.

Validación inicial: Evita errores si la función es llamada sin datos

```
if not datos_venta:  
    print("No se proporcionaron datos para generar la factura.")  
    return None
```

Preparar la ruta del archivo

- Verifica si existe la carpeta facturas/; si no, la crea.
- Define el nombre del archivo según el número de factura.

```
# Define el nombre del archivo  
numero_factura = datos_venta['venta_info']['numero_factura']  
# Crear la carpeta 'facturas' si no existe  
if not os.path.exists('facturas'):  
    os.makedirs('facturas')  
file_path = os.path.join('facturas', f"{numero_factura}.pdf")
```

Crear el documento base

Crea el documento base y una lista elements que contendrá los bloques (párrafos, tablas, etc.).

```
doc = SimpleDocTemplate(file_path, pagesize=letter)
elements = []
styles = getSampleStyleSheet()
```

Información del cliente

Sección con los datos del comprador, formateados de manera profesional.

```
# --- Datos del Cliente ---
cliente = datos_venta['cliente_info']
elements.append(Paragraph("<b>Facturado a:</b>", styles['h3']))
elements.append(Paragraph(f"{cliente['nombre']} {cliente['apellido']}", styles['Normal']))
elements.append(Paragraph(f"Documento: {cliente['num_doc']}", styles['Normal']))
elements.append(Paragraph(f"Dirección: {cliente['direccion']}", styles['Normal']))
elements.append(Paragraph(f"Correo: {cliente['correo']}", styles['Normal']))
elements.append(Paragraph("<br/><br/>", styles['Normal']))
```

Generación final del PDF

```
# Construir el PDF
doc.build(elements)
return file_path
```

9. Ventana bd

Este módulo contiene la clase **Database**, encargada de **gestionar la conexión con la base de datos MySQL** del sistema de ventas, llamada **ventum**.

A través de esta clase, la aplicación puede:

- Conectarse de forma segura al servidor MySQL.
- Ejecutar consultas SQL (como leer productos, registrar ventas, etc.).
- Cerrar la conexión correctamente al finalizar el uso.

Función: connect()

Esta función:

1. Establece la conexión con el servidor MySQL local (localhost).
2. Usa las credenciales:
 - Usuario: root
 - Contraseña: 1234

○ Base de datos: ventum

3. Verifica si la conexión fue exitosa y muestra un mensaje en la consola.
4. En caso de error (por ejemplo, contraseña incorrecta o servidor inactivo), muestra el error y evita que el programa falle.

```
def connect(self):  
    """  
    Establece la conexión con la base de datos.  
    """  
    try:  
        self.connection = mysql.connector.connect(  
            host="localhost",  
            user="root",  
            password="",  
            database="ventum"  
        )  
        if self.connection.is_connected():  
            print("Conexión a la base de datos establecida con éxito.")  
    except mysql.connector.Error as e:  
        print(f"Error al conectar a la base de datos: {e}")  
        self.connection = None
```

Función: close_connection()

Cierra correctamente la conexión activa con la base de datos, liberando los recursos utilizados.

Es una buena práctica llamar a esta función al salir del programa o al finalizar las operaciones que requieren acceso a los datos.

```
def close_connection(self):  
    """  
    Cierra la conexión con la base de datos.  
    """  
    if self.connection and self.connection.is_connected():  
        self.connection.close()  
        print("Conexión a la base de datos cerrada.")
```

Función: obtener_productos()

Esta función se encarga de **consultar la tabla producto** en la base de datos y devolver una lista de tuplas con la información de cada producto.

Pasos principales:

1. Verifica que exista una conexión activa.
2. Crea un **cursor** para ejecutar comandos SQL.
3. Ejecuta la consulta SELECT para obtener todos los productos.
4. Retorna los resultados en forma de lista de tuplas.

En caso de error, muestra el mensaje correspondiente y
retorna una lista vacía.

```
def obtener_productos(self):  
    """  
    Obtiene todos los productos de la tabla 'producto'.  
    Retorna una lista de tuplas con los datos de los productos.  
    """  
    if not self.connection or not self.connection.is_connected():  
        print("Error: No hay una conexión activa a la base de datos.")  
        return []  
  
    cursor = self.connection.cursor()  
    query = "SELECT id_producto, categoria, marca, nombre, descripcion, precio, stock FROM producto"  
    try:  
        cursor.execute(query)  
        productos = cursor.fetchall()  
        return productos  
    except mysql.connector.Error as e:  
        print(f"Error al ejecutar la consulta: {e}")  
        return []  
    finally:  
        cursor.close()
```

10. Models

a. Venta.py

Este módulo se encarga de **administrar las operaciones**

relacionadas con las ventas, incluyendo el **registro de nuevas ventas**, la **actualización del stock**, y la **consulta de ventas registradas** en la base de datos.

Además, provee funciones para listar los **tipos de pago disponibles**, utilizando la clase Database del módulo db.py para conectarse con el sistema MySQL.

Importaciones y dependencias

- **from db import Database:** importa la clase encargada de gestionar la conexión a la base de datos.
- **from mysql.connector import Error:** permite manejar los errores que puedan ocurrir durante las operaciones con la base de datos.

```
from db import Database
from mysql.connector import Error
```

listar_ventas()

- Crea una conexión a la base de datos mediante Database().
- Usa un cursor tipo diccionario para obtener los resultados con nombres de columnas.
- Ejecuta una consulta SQL con varias uniones (INNER JOIN) entre las tablas:

- venta
- cliente
- detalle_venta
- producto

-Devuelve todas las filas ordenadas por fecha descendente.

-Cierra el cursor y la conexión al finalizar.

```
def listar_ventas():
    """
    Lista todas las ventas con el nombre del cliente y el nombre del producto.
    """
    db = Database()
    conn = db.connection
    if conn and conn.is_connected():
        try:
            cursor = conn.cursor(dictionary=True)
            cursor.execute("""
                SELECT
                    v.id_venta,
                    c.nombre AS nombre_cliente,
                    c.apellido AS apellido_cliente,
                    p.nombre AS nombre_producto,
                    v.fecha,
                    v.total,
                    dv.cantidad
                FROM venta v
                INNER JOIN cliente c ON v.fk_id_cliente = c.id_cliente
                INNER JOIN detalle_venta dv ON v.id_venta = dv.fk_id_venta
                INNER JOIN producto p ON dv.fk_id_producto = p.id_producto
                ORDER BY v.fecha DESC;
            """)
            rows = cursor.fetchall()
            return rows
        except Error as e:
            print(f"Error al listar ventas: {e}")
            return []
        finally:
            if 'cursor' in locals() and cursor:
                cursor.close()
            db.close_connection()
    else:
        print("Error: No se pudo conectar a la base de datos.")
        return []
```

registrar_venta()

1. Se establece la conexión con la base de datos.
2. Se calculan los valores numéricos:
3. Se determina el estado de pago según si existe saldo pendiente:
4. Se insertan los registros correspondientes:

- a. **Tabla venta** (datos generales de la venta).
 - b. **Tabla detalle_venta** (detalle de productos vendidos).
 - c. Se actualiza el stock en la **tabla producto**.
5. Finalmente, se confirma la transacción con `conn.commit()` para guardar los cambios.
 6. En caso de error, se revierte la operación con `conn.rollback()` para mantener la integridad de los datos.

```
def registrar_venta(cliente_id, tipo_pago, producto_id, cantidad, precio_unitario, saldo_pendiente=0, descuento=0, observaciones=None):
    """
    Registra una nueva venta y actualiza el stock del producto.
    """
    db = Database()
    conn = db.connection

    if conn and conn.is_connected():
        try:
            cursor = conn.cursor()

            subtotal = cantidad * precio_unitario
            total = subtotal - descuento
            impuesto = int(total * 0.19)

            # Asignar el ID de estado de pago basado en el saldo pendiente
            # 2 = 'Pendiente'
            # 1 = 'Pagado'
            estado_pago = 2 if saldo_pendiente > 0 else 1

            cursor.execute("""
                INSERT INTO venta (fk_id_cliente, fk_id_tipo_pago, subtotal, descuento, impuesto, total, saldo_pendiente, fk_id_estado_pago, observaciones)
                VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)
            """, (cliente_id, tipo_pago, subtotal, descuento, impuesto, total, saldo_pendiente, estado_pago, observaciones))
            venta_id = cursor.lastrowid

            cursor.execute("""
                INSERT INTO detalle_venta (fk_id_venta, fk_id_producto, cantidad, precio_unitario)
                VALUES (%s,%s,%s,%s)
            """, (venta_id, producto_id, cantidad, precio_unitario))

            cursor.execute("""
                UPDATE producto SET stock = stock - %s WHERE id_producto=%s
            """, (cantidad, producto_id))

            conn.commit()
            return venta_id
        except Error as e:
            print(f"Error al registrar venta: {e}")
            conn.rollback()
            return None
        finally:
            cursor.close()
            db.close_connection()
    else:
        print("Error: No se pudo conectar a la base de datos.")
        return None
```

listar_tipos_pago()

1. Crea una conexión mediante la clase Database.
2. Ejecuta la consulta SQL:
3. Devuelve los resultados como una lista de diccionarios.
4. Cierra el cursor y la conexión.

```
def listar_tipos_pago():  
    """  
    Lista todos los tipos de pago disponibles.  
    """  
    db = Database()  
    conn = db.connection  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor(dictionary=True)  
            cursor.execute("SELECT id_tipo_pago, nombre FROM tipo_pago")  
            tipos_pago = cursor.fetchall()  
            return tipos_pago  
        except Error as e:  
            print(f"Error al listar tipos de pago: {e}")  
            return []  
        finally:  
            if 'cursor' in locals() and cursor:  
                cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")  
        return []
```

b. Usuario.py

Este módulo implementa la función encargada de **verificar las credenciales de acceso** del usuario al sistema.

Su objetivo principal es garantizar que solo los usuarios registrados y activos puedan iniciar sesión correctamente.

Utiliza la clase Database (definida en el módulo db.py) para conectarse a la base de datos MySQL y validar los datos ingresados.

- Database: clase encargada de gestionar la conexión a la base de datos.
- Error: clase de excepción que permite capturar errores durante la conexión o ejecución de consultas SQL.

```
from db import Database
from mysql.connector import Error
```

validar_usuario()

Verificar si el nombre de usuario y la contraseña ingresados coinciden con un registro existente en la base de datos y que dicho usuario se encuentre activo.

Esta validación es esencial para el control de acceso al sistema, ya que protege los módulos internos (ventas, inventario, reportes, etc.) de accesos no autorizados.

- **Conexión a la base de datos**

Se crea una instancia de la clase Database y se obtiene la conexión activa:

- **Verificación de conexión activa**

Antes de ejecutar la consulta, se comprueba que la conexión esté establecida correctamente:

- **Ejecución de la consulta SQL**

Si la conexión está activa, se crea un cursor con formato de diccionario (dictionary=True) para que los resultados sean devueltos con nombres de columnas.

- La consulta valida que el usuario exista, la contraseña coincida y que el campo activo sea igual a 1 (es decir, que el usuario esté habilitado para ingresar):

- **Obtención del resultado**

- Si las credenciales son correctas, retorna un diccionario con los datos del usuario (por ejemplo: ID, nombre, rol, etc.).

Si no coinciden, retorna None.

- **Manejo de errores**

En caso de fallar la conexión o la ejecución de la consulta, se captura la excepción:

- **Cierre de recursos**

El cursor y la conexión se cierran siempre, ya sea que el proceso tenga éxito o

no:

```
def validar_usuario(usuario, clave):  
    """  
    Valida las credenciales de un usuario.  
    Retorna la fila del usuario si las credenciales son correctas, de lo contrario, None.  
    """  
    db = Database()  
    conn = db.connection  
  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor(dictionary=True)  
            # Se corrigen los nombres de las columnas a 'usuario' y 'clave'  
            cursor.execute("""  
                SELECT * FROM usuario WHERE usuario=%s AND clave=%s AND activo=1  
            """, (usuario, clave))  
  
            row = cursor.fetchone()  
            return row  
  
        except Error as e:  
            print(f"Error al validar usuario: {e}")  
            return None  
        finally:  
            cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")  
        return None
```

c. Reportes.py

El módulo reportes.py agrupa un conjunto de **funciones de consulta y análisis de datos** sobre las tablas principales del sistema de ventas (producto, cliente, venta, detalle_venta).

Su propósito es generar **informes estadísticos** que permiten al administrador o encargado del sistema obtener una visión clara sobre las operaciones realizadas, los clientes más activos y los productos más vendidos.

productos_mas_vendidos()

Obtiene una lista de los 10 productos con mayor cantidad de ventas, sumando las unidades vendidas en todas las transacciones registradas.

Conexión con la base de datos

Se inicia una conexión activa usando la clase Database.

Ejecución de la consulta SQL

- Se realiza una unión entre las tablas **producto** y **detalle_venta**.
- Se agrupan los resultados por nombre de producto.
- Se ordenan de forma descendente según la cantidad total vendida.
- Se limita el resultado a los 10 primeros productos.

Retorno del resultado

Devuelve una lista de diccionarios con los nombres y cantidades vendidas.

```
def productos_mas_vendidos():
    """
    Lista los productos más vendidos por cantidad.
    """
    db = Database()
    conn = db.connection
    if conn and conn.is_connected():
        try:
            cursor = conn.cursor(dictionary=True)
            cursor.execute("""
                SELECT
                    p.nombre,
                    SUM(dv.cantidad) AS total_vendido
                FROM producto p
                INNER JOIN detalle_venta dv ON p.id_producto = dv.fk_id_producto
                GROUP BY p.nombre
                ORDER BY total_vendido DESC
                LIMIT 10;
            """)
            rows = cursor.fetchall()
            return rows
        except Error as e:
            print(f"Error al obtener productos más vendidos: {e}")
            return []
        finally:
            if 'cursor' in locals() and cursor:
                cursor.close()
            db.close_connection()
    else:
        print("Error: No se pudo conectar a la base de datos.")
        return []
```

clientes_frecuentes()

Obtiene los 10 clientes más activos en el sistema, según la cantidad total de ventas realizadas.

1. Se vinculan las tablas cliente y venta.
2. Se cuentan las ventas asociadas a cada cliente.
3. El resultado se ordena de mayor a menor cantidad de compras.

```
def clientes_frecuentes():
    """
    Lista los clientes que han realizado más compras.
    """
    db = Database()
    conn = db.connection
    if conn and conn.is_connected():
        try:
            cursor = conn.cursor(dictionary=True)
            cursor.execute("""
                SELECT
                    c.nombre,
                    c.apellido,
                    COUNT(v.id_venta) AS compras
                FROM cliente c
                INNER JOIN venta v ON c.id_cliente = v.fk_id_cliente
                GROUP BY c.id_cliente
                ORDER BY compras DESC
                LIMIT 10;
            """)
            rows = cursor.fetchall()
            return rows
        except Error as e:
            print(f"Error al obtener clientes frecuentes: {e}")
            return []
        finally:
            if 'cursor' in locals() and cursor:
                cursor.close()
            db.close_connection()
    else:
        print("Error: No se pudo conectar a la base de datos.")
        return []
```

saldos_pendientes()

Permite generar un informe de los clientes con pagos incompletos o deudas activas, mostrando su nombre, apellido y el saldo pendiente.

1. Filtra las ventas cuyo estado de pago (fk_id_estado_pago) es **2**, es decir, **pendiente**.
2. Solo se incluyen los casos donde el campo saldo_pendiente es mayor que cero.

```
def saldos_pendientes():  
    """  
    Lista los clientes con saldo pendiente en sus ventas.  
    """  
    db = Database()  
    conn = db.connection  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor(dictionary=True)  
            cursor.execute("""  
                SELECT  
                    c.nombre,  
                    c.apellido,  
                    v.saldo_pendiente  
                FROM venta v  
                INNER JOIN cliente c ON v.fk_id_cliente = c.id_cliente  
                WHERE v.fk_id_estado_pago = 2 AND v.saldo_pendiente > 0;  
            """)  
            rows = cursor.fetchall()  
            return rows  
        except Error as e:  
            print(f"Error al obtener saldos pendientes: {e}")  
            return []  
        finally:  
            if 'cursor' in locals() and cursor:  
                cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")  
        return []
```

liquidaciones_ventas()

Genera un resumen contable de las ventas agrupadas por fecha, mostrando el total monetario obtenido cada día.

1. Se usa la función DATE() para obtener solo la parte de fecha.
2. Se agrupan los registros de la tabla venta por fecha.
3. Se calcula la suma total de cada día.

```
def liquidaciones_ventas():
    """
    Calcula el total de ventas por día.
    """
    db = Database()
    conn = db.connection
    if conn and conn.is_connected():
        try:
            cursor = conn.cursor(dictionary=True)
            cursor.execute("""
                SELECT
                    DATE(fecha) AS fecha_venta,
                    SUM(total) AS total_diario
                FROM venta
                GROUP BY fecha_venta
                ORDER BY fecha_venta DESC;
            """)
            rows = cursor.fetchall()
            return rows
        except Error as e:
            print(f"Error al obtener liquidaciones de ventas: {e}")
            return []
        finally:
            if 'cursor' in locals() and cursor:
                cursor.close()
            db.close_connection()
    else:
        print("Error: No se pudo conectar a la base de datos.")
        return []
```

d. Productos.py

El módulo productos.py centraliza todas las operaciones relacionadas con la **tabla producto** de la base de datos ventum.

Permite **consultar, agregar y actualizar** los productos del inventario, incluyendo su información principal, estado y stock disponible.

listar_productos()

Esta función devuelve todos los productos que están activos (estado = 1) en la base de datos.

Se utiliza, por ejemplo, para mostrar el inventario disponible en la interfaz del sistema.

```
def listar_productos():  
    """  
    Lista todos los productos activos de la base de datos.  
    """  
    db = Database()  
    conn = db.connection  
  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor(dictionary=True)  
            cursor.execute("SELECT * FROM producto WHERE estado = 1")  
            rows = cursor.fetchall()  
            return rows  
        except Error as e:  
            print(f"Error al listar productos: {e}")  
            return []  
        finally:  
            cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")  
        return []
```

agregar_producto()

Permite insertar un nuevo producto en la base de datos, con todos sus atributos y estado inicial (activo por defecto).

1. Se reciben los parámetros del producto desde el formulario o backend.
2. Se ejecuta la sentencia INSERT mediante el cursor.
3. Se confirma la transacción con `conn.commit()`.
4. Si ocurre un error, se hace `conn.rollback()` para revertir los cambios.

```
def agregar_producto(categoria, marca, nombre, descripcion, precio, stock, imagen, estado=True):
    """
    Agrega un nuevo producto a la base de datos.
    """
    db = Database()
    conn = db.connection

    if conn and conn.is_connected():
        try:
            cursor = conn.cursor()
            cursor.execute("""
                INSERT INTO producto (categoria, marca, nombre, descripcion, precio, stock, imagen, estado)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (categoria, marca, nombre, descripcion, precio, stock, imagen, estado))
            conn.commit()
            print("Producto agregado con éxito.")
        except Error as e:
            print(f"Error al agregar producto: {e}")
            conn.rollback() # Deshacer cambios si ocurre un error
        finally:
            cursor.close()
            db.close_connection()
    else:
        print("Error: No se pudo conectar a la base de datos.")
```

actualizar_stock()

Modifica la cantidad de stock disponible de un producto específico dentro de la base de datos.

Esta función es fundamental para mantener actualizado el inventario después de cada venta.

1. Se actualiza **solo el campo stock** del producto indicado por su ID.
2. Se confirma el cambio con `conn.commit()`.
3. En caso de error, se ejecuta `conn.rollback()` para evitar inconsistencias.

```
def actualizar_stock(id_producto, nuevo_stock):  
    """  
    Actualiza el stock de un producto.  
    """  
    db = Database()  
    conn = db.connection  
  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor()  
            cursor.execute("UPDATE producto SET stock=%s WHERE id_producto=%s", (nuevo_stock, id_producto))  
            conn.commit()  
            print("Stock actualizado con éxito.")  
        except Error as e:  
            print(f"Error al actualizar stock: {e}")  
            conn.rollback() # Deshacer cambios si ocurre un error  
        finally:  
            cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")
```

e. Clientes.py

El módulo `clientes.py` se encarga de administrar la información de los clientes registrados en el sistema.

Contiene las funciones necesarias para **listar los clientes activos** y **registrar nuevos clientes** en la base de datos MySQL.

Está estrechamente relacionado con la tabla cliente y el tipo de documento (tipo_doc) en la base de datos ventum.

listar_clientes()

Recupera todos los registros de clientes que estén activos (activo = 1) y los devuelve como una lista de diccionarios.

También muestra el tipo de documento de cada cliente mediante una relación con la tabla tipo_doc.

1. Una **unión interna (INNER JOIN)** entre las tablas cliente y tipo_doc.
2. Devuelve todos los campos del cliente (c.*) más las **siglas del tipo de documento** (t.siglas).
3. Filtra únicamente los clientes activos (c.activo = 1).

Flujo de ejecución

1. Se crea la conexión con la base de datos usando Database().
2. Se abre un cursor en formato diccionario (dictionary=True).
3. Se ejecuta la consulta SQL para listar los clientes activos.
4. Se almacenan los resultados con fetchall().
5. Se devuelven los resultados a la interfaz o módulo que los solicitó.

6. Se cierran el cursor y la conexión.

```
def listar_clientes():  
    """  
    Lista todos los clientes activos de la base de datos.  
    """  
    db = Database()  
    conn = db.connection  
  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor(dictionary=True)  
            cursor.execute("""  
                SELECT c.*, t.siglas  
                FROM cliente c  
                INNER JOIN tipo_doc t ON c.fk_id_tipo_doc = t.id_tipo_doc  
                WHERE c.activo = 15  
            """)  
            rows = cursor.fetchall()  
            return rows  
        except Error as e:  
            print(f"Error al listar clientes: {e}")  
            return []  
        finally:  
            cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")  
        return []
```

agregar_cliente()

Inserta un nuevo cliente en la base de datos ventum con todos sus datos personales y de contacto.

Además, establece el campo activo en True por defecto para indicar que el cliente está habilitado en el sistema.

Funcionamiento paso a paso

1. Conexión: Se crea una instancia de Database y se obtiene la conexión.
2. Ejecución: Se envían los parámetros con valores concretos.
3. Confirmación: La transacción se guarda con conn.commit().
4. Seguridad: Los valores se pasan de forma parametrizada (%s) para evitar inyección SQL.
5. Recuperación ante errores: Si algo falla, se ejecuta conn.rollback() para revertir los cambios.
6. Cierre: Se cierran el cursor y la conexión.

```
def agregar_cliente(fk_id_tipo_doc, nombre, apellido, correo, direccion, metodo_pago, ciudad, departamento, num_doc, activo=True):  
    """  
    Agrega un nuevo cliente a la base de datos.  
    """  
    db = Database()  
    conn = db.connection  
  
    if conn and conn.is_connected():  
        try:  
            cursor = conn.cursor()  
            cursor.execute("""  
                INSERT INTO cliente (fk_id_tipo_doc, nombre, apellido, correo, direccion, metodo_pago, ciudad, departamento, num_doc, activo)  
                VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)  
            """, (fk_id_tipo_doc, nombre, apellido, correo, direccion, metodo_pago, ciudad, departamento, num_doc, activo))  
            conn.commit()  
            print("Cliente agregado con éxito.")  
        except Error as e:  
            print(f"Error al agregar cliente: {e}")  
            conn.rollback() # Deshacer cambios si ocurre un error  
        finally:  
            cursor.close()  
            db.close_connection()  
    else:  
        print("Error: No se pudo conectar a la base de datos.")
```

Este módulo está diseñado para manejar la **lógica de las cuentas de crédito de los clientes**, incluyendo la creación automática de cuentas, la actualización de saldos y el registro de abonos relacionados con ventas.

El código se conecta a la base de datos a través de una clase auxiliar llamada Database, utilizando el conector oficial de **MySQL para Python (mysql.connector)**.

Se aplican prácticas de manejo de errores y transacciones para garantizar la integridad de los datos.

listar_planes_credito()

Obtiene todos los planes de crédito activos registrados en la base de datos.

Se usa principalmente para **mostrar opciones de crédito disponibles** al momento de registrar una venta o crear una cuenta.

1. Conecta con la base de datos usando la clase Database.
2. Ejecuta la consulta SQL:
3. Retorna una lista de diccionarios con los resultados.

```
def listar_planes_credito():  
    """  
    Obtiene todos los planes de crédito disponibles desde la base de datos.  
    """  
    db = Database()  
    conn = db.connection  
    if not (conn and conn.is_connected()):  
        return []  
    try:  
        cursor = conn.cursor(dictionary=True)  
        cursor.execute("SELECT id_plan_credito, nombre FROM plan_credito WHERE activo = 1")  
        return cursor.fetchall()  
    except Error as e:  
        print(f"Error al listar planes de crédito: {e}")  
        return []  
    finally:  
        if conn.is_connected():  
            cursor.close()  
            db.close_connection()
```

obtener_o_crear_cuenta_credito(id_cliente)

Verifica si un cliente ya tiene una cuenta de crédito registrada.

Si **no existe**, crea una nueva automáticamente.

1. Buscar cuenta existente:
2. Si no existe, obtener los datos del cliente:
3. Generar código único para la cuenta:
Se forma con las primeras letras del nombre y apellido, seguido del ID.
4. Crear la cuenta de crédito con cupo inicial:

```
def obtener_o_crear_cuenta_credito(id_cliente):
    """
    Busca la cuenta de crédito de un cliente. Si no existe, la crea.
    Retorna el ID de la cuenta.
    """
    db = Database()
    conn = db.connection
    if not (conn and conn.is_connected()):
        return None

    try:
        cursor = conn.cursor(dictionary=True)
        # 1. Buscar cuenta existente
        cursor.execute("SELECT id_cuenta FROM cuenta_credito WHERE fk_id_cliente = %s", (id_cliente,))
        cuenta = cursor.fetchone()
        if cuenta:
            return cuenta['id_cuenta']

        # 2. Si no existe, crear una nueva
        # Generar un código único para la cuenta
        cursor.execute("SELECT nombre, apellido FROM cliente WHERE id_cliente = %s", (id_cliente,))
        cliente_info = cursor.fetchone()
        codigo_cuenta = f"CR-{cliente_info['nombre'][:2].upper()}{cliente_info['apellido'][:2].upper()}-{id_cliente}"

        # Cupo máximo por defecto, podría ser configurable en el futuro
        cupo_maximo = 5000000

        cursor.execute(
            "INSERT INTO cuenta_credito (fk_id_cliente, codigo, saldo_actual, cupo_maximo, estado) VALUES (%s, %s, 0, %s, 1)",
            (id_cliente, codigo_cuenta, cupo_maximo)
        )
        id_nueva_cuenta = cursor.lastrowid
        conn.commit()
        return id_nueva_cuenta

    except Error as e:
        print(f"Error al obtener o crear cuenta de crédito: {e}")
        conn.rollback()
        return None
    finally:
        if conn.is_connected():
            cursor.close()
            db.close_connection()
```

actualizar_saldo_cuenta(id_cuenta, monto_a_sumar)

Actualiza el **saldo actual** (deuda o crédito disponible) de una cuenta de crédito, sumando el monto indicado.

```
def actualizar_saldo_cuenta(id_cuenta, monto_a_sumar):  
    """  
    Añade un monto al saldo actual (deuda) de una cuenta de crédito.  
    """  
    db = Database()  
    conn = db.connection  
    if not (conn and conn.is_connected()):  
        return  
    try:  
        cursor = conn.cursor()  
        cursor.execute("UPDATE cuenta_credito SET saldo_actual = saldo_actual + %s WHERE id_cuenta = %s", (monto_a_sumar, id_cuenta))  
        conn.commit()  
    except Error as e:  
        print(f"Error al actualizar saldo de cuenta de crédito: {e}")  
        conn.rollback()  
    finally:  
        if conn.is_connected():  
            cursor.close()  
            db.close_connection()
```

buscar_venta_por_id(venta_id)

Busca una venta específica en la base de datos y obtiene también los datos del cliente asociado.

Esto se usa, por ejemplo, al momento de realizar un **abono**.

```
def buscar_venta_por_id(venta_id):
    """
    Busca una venta específica y los datos del cliente asociado.
    """
    db = Database()
    conn = db.connection
    if not (conn and conn.is_connected()):
        return None
    try:
        cursor = conn.cursor(dictionary=True)
        cursor.execute("""
            SELECT v.id_venta, v.saldo_pendiente, v.fk_id_cliente, c.nombre AS nombre_cliente, c.apellido AS apellido_cliente
            FROM venta v
            JOIN cliente c ON v.fk_id_cliente = c.id_cliente
            WHERE v.id_venta = %s
        """, (v, (variable) cursor: MySQLCursorAbstract | Any
        return cursor.fetchone()
    except Error as e:
        print(f"Error al buscar venta por ID: {e}")
        return None
    finally:
        if conn.is_connected():
            cursor.close()
            db.close_connection()
```

registrar_abono_a_venta(datos_abono)

Registra un abono (pago parcial) a una venta y actualiza el saldo pendiente.
Garantiza la consistencia de datos utilizando transacciones.

1. Inicia una transacción:
2. Actualiza el saldo pendiente:
3. Registra el abono en la tabla correspondiente:
4. Confirma la transacción:

```
def registrar_abono_a_venta(datos_abono):
    """
    Registra un abono y actualiza el saldo pendiente de la venta.
    Utiliza una transacción para garantizar la integridad.
    """
    db = Database()
    conn = db.connection
    if not (conn and conn.is_connected()):
        return False

    try:
        cursor = conn.cursor()
        conn.start_transaction()

        # 1. Actualizar el saldo_pendiente en la tabla 'venta'
        cursor.execute(
            "UPDATE venta SET saldo_pendiente = saldo_pendiente - %s WHERE id_venta = %s",
            (datos_abono['monto_abonado'], datos_abono['fk_id_venta'])
        )

        # 2. Insertar el registro en la tabla 'abono'
        query_abono = """
        INSERT INTO abono (fk_id_venta, fk_id_cliente, fk_id_usuario, monto_abonado, metodo_pago, observaciones)
        VALUES (%s, %s, %s, %s, %s, %s)
        """
        params_abono = (datos_abono['fk_id_venta'], datos_abono['fk_id_cliente'], datos_abono['fk_id_usuario'], datos_abono['monto_abonado'], datos_abono['metodo_pago'], datos_abono['observaciones'])
        cursor.execute(query_abono, params_abono)

        conn.commit()
        return True
    except Error as e:
        print(f"Error al registrar abono: {e}")
        conn.rollback()
        return False
    finally:
        if conn.is_connected():
            cursor.close()
            db.close_connection()
```

11. Logica_iu

a. Registro usuario.py

Este módulo implementa una ventana de registro de usuarios dentro de una aplicación gráfica desarrollada con PyQt6.

Permite capturar los datos personales y de acceso del nuevo usuario, validarlos y registrarlos en la base de datos mediante la función registrar_usuario() del modelo correspondiente.

La interfaz está construida en el archivo .ui llamado register_usuario_fixed.ui, que contiene los campos de entrada y botones necesarios para la interacción con el usuario.

Carga la interfaz y conecta el botón de registro con la función crear_usuario().

```
class RegistroUsuarioWindow(QtWidgets.QDialog):  
    def __init__(self):  
        super().__init__()  
        uic.loadUi("ui/register_usuario_fixed.ui", self)  
        self.pushButton_register.clicked.connect(self.crear_usuario)
```

Obtiene el texto de los campos (nombre, apellido, usuario, correo, clave, confirmar_clave) con .text().

```
def crear_usuario(self):  
    nombre = self.lineEdit_name.text()  
    apellido = self.lineEdit_apellido.text()  
    usuario = self.lineEdit_usuario.text()  
    correo = self.lineEdit_email.text()  
    clave = self.lineEdit_password.text()  
    confirmar_clave = self.lineEdit_confirm.text()
```

Verifica:

- Que todos los campos estén completos.
- Que las contraseñas coincidan.
- Que la contraseña tenga al menos 8 caracteres.

Si falla, muestra un mensaje de advertencia con QMessageBox.

```
# Validaciones básicas  
if not all([nombre, apellido, usuario, correo, clave, confirmar_clave]):  
    QtWidgets.QMessageBox.warning(self, "Error", "Todos los campos son obligatorios.")  
    return  
  
if clave != confirmar_clave:  
    QtWidgets.QMessageBox.warning(self, "Error", "Las contraseñas no coinciden.")  
    return  
  
if len(clave) < 8:  
    QtWidgets.QMessageBox.warning(self, "Error", "La contraseña debe tener al menos 8 caracteres.")  
    return
```

Llama a registrar_usuario().

Si tiene éxito, muestra “Usuario registrado correctamente” y cierra la ventana.

Si falla, muestra un error.

```
# Llamar a la función del modelo para registrar
# Por defecto, se registrará como 'CLIENTE'
if registrar_usuario(nombre, apellido, usuario, clave):
    QtWidgets.QMessageBox.information(self, "Éxito", "Usuario registrado correctamente.")
    self.close()
else:
    QtWidgets.QMessageBox.critical(self, "Error", "No se pudo registrar el usuario. Es posible que el nombre de usuario ya esté en uso.")
```

b. Registro cliente.py

Esta clase gestiona el registro de clientes en la aplicación usando PyQt6.

Se basa en la interfaz register_client_with_fiador.ui y permite ingresar los datos personales y de contacto del cliente para guardarlos en la base de datos.

Inicializa la ventana, configura la interfaz con setupUi(self) y conecta el botón **Registrar** con la función registrar_cliente().

Además, carga los tipos de documento en el ComboBox mediante cargar_tipos_doc().

```
class RegistroClienteWindow(QtWidgets.QMainWindow, Ui_RegisterClientWindow):
    def __init__(self):
        super().__init__()
        # Usamos setupUi para configurar la interfaz
        self.setupUi(self)

        # Conecta el botón 'SubmitButton' a la función de registro
        self.SubmitButton.clicked.connect(self.registrar_cliente)

        # Carga (function) cargar_tipos_doc: Any ComboBox
        self.cargar_tipos_doc()
```

registrar_cliente()

Captura la información escrita en los campos del formulario (nombre, documento, correo, dirección, etc.).

Separa nombre y apellido automáticamente y llama a la función agregar_cliente() del modelo para guardar el registro.

Si el proceso es exitoso, muestra un mensaje de confirmación.

```
def registrar_cliente(self):
    try:
        # Lee los datos de todos los campos del formulario
        nombre_completo = self.InputNombre.text()
        correo = self.InputCorreo.text()
        tipo_doc_id = self.ComboTipo.currentData()
        num_doc = self.InputNumero.text()
        telefono = self.InputTelefono.text()
        celular = self.InputDireccion.text() # Este campo es 'Celular' en la UI
        direccion = self.InputOtros.text() # Este campo es 'Direccion' en la UI
        codigo = self.InputCodigo.text()
        fiadora = self.InputFiadora.text()
        codigo_fiador = self.InputCodigoFiador.text()

        # Separar nombre y apellido (asumiendo el primer nombre y el resto como apellido)
        partes_nombre = nombre_completo.split(' ', 1)
        nombre = partes_nombre[0]
        apellido = partes_nombre[1] if len(partes_nombre) > 1 else ""

        # Llama a la función del modelo para registrar el cliente
        # Nota: La función agregar_cliente puede necesitar ajustes para aceptar todos estos campos.
        agregar_cliente(
            fk_id_tipo_doc=tipo_doc_id, # Asegúrate que tu función lo use
            nombre=nombre,
            apellido=apellido,
            correo=correo,
            direccion=direccion,
            num_doc=num_doc,
            # Los siguientes son valores por defecto, ya que no están en el formulario
            metodo_pago="Efectivo",
            ciudad="Medellin",
            departamento="Antioquia"
        )
        QtWidgets.QMessageBox.information(self, "Éxito", "Cliente registrado con éxito.")
        self.close()
    except Exception as e:
        QtWidgets.QMessageBox.warning(self, "Error", f"No se pudo registrar el cliente: {e}")
```

cargar_tipos_doc()

Agrega al ComboBox una lista de tipos de documentos predefinidos (CC, CE, Pasaporte).

Esta lista puede venir de la base de datos en versiones futuras.

```
def cargar_tipos_doc(self):
    # Idealmente, esto vendría de la base de datos.
    tipos_doc = [
        {"id_tipo_doc": 1, "siglas": "CC"},
        {"id_tipo_doc": 2, "siglas": "CE"},
        {"id_tipo_doc": 3, "siglas": "Pasaporte"}
    ]

    self.ComboTipo.clear()
    self.ComboTipo.addItem("Seleccionar", -1) # Opción por defecto
    for doc in tipos_doc:
        self.ComboTipo.addItem(doc["siglas"], doc["id_tipo_doc"])
```

c. Registrar usuario window.py

Esta clase crea una ventana emergente de registro de usuario usando PyQt6.

Se basa en la interfaz register_usuario_fixed_ui.ui y permite mostrar los campos de registro definidos en la interfaz gráfica.

Constructor (__init__)

Inicializa el diálogo y configura la interfaz con el método setupUi(self).

Esto carga todos los elementos del archivo .ui en la ventana, dejándola lista para interactuar con el usuario.

```
class RegistrarUsuarioWindow(QtWidgets.QDialog):
    def __init__(self):
        super().__init__()
        self.ui = Ui_FormRegistrarUsuario()
        self.ui.setupUi(self)
```

d. Registrar abono.py

Esta clase crea una ventana para registrar pagos parciales (abonos) a ventas a crédito, conectando la interfaz de PyQt6 con las funciones del modelo de base de datos.

cargar_info_venta()

Busca una venta por ID y muestra la información del cliente y el saldo pendiente.

```
def cargar_info_venta(self):  
    """  
    Busca la venta por ID y carga la información en la UI.  
    """  
    try:  
        venta_id = int(self.lineEdit_invoiceNumberToPay.text())  
    except ValueError:  
        self.limpiar_campos()  
        return  
  
    self.venta_actual = buscar_venta_por_id(venta_id)  
  
    if self.venta_actual and self.venta_actual.get('saldo_pendiente', 0) > 0:  
        cliente_nombre = f"{self.venta_actual['nombre_cliente']} {self.venta_actual['apellido_cliente']}"  
        saldo_actual = self.venta_actual['saldo_pendiente']  
  
        self.lineEdit_accountName.setText(cliente_nombre)  
        self.doubleSpinBox_remaining.setValue(saldo_actual)  
        self.doubleSpinBox_paid.setMaximum(saldo_actual)  
        self.doubleSpinBox_paid.setValue(0)  
    else:  
        QtWidgets.QMessageBox.warning(self, "Venta no encontrada", "No se encontró una venta a crédito con ese ID o ya fue pagada.")  
        self.limpiar_campos()
```

calcular_saldo_restante()

Actualiza el saldo restante cuando se ingresa un monto a abonar.

```
def calcular_saldo_restante(self):  
    if self.venta_actual:  
        saldo_anterior = self.venta_actual['saldo_pendiente']  
        monto_abonado = self.doubleSpinBox_paid.value()  
        nuevo_saldo = saldo_anterior - monto_abonado  
        self.doubleSpinBox_remaining.setValue(nuevo_saldo)
```

guardar_abono()

Guarda el pago en la base de datos, validando que el monto sea válido y la venta exista.

```
def guardar_abono(self):
    if not self.venta_actual:
        QtWidgets.QMessageBox.warning(self, "Sin Venta", "Por favor, busque una venta válida antes de registrar un abono.")
        return

    monto_abonado = self.doubleSpinBox_paid.value()
    if monto_abonado <= 0:
        QtWidgets.QMessageBox.warning(self, "Monto Inválido", "El monto a abonar debe ser mayor que cero.")
        return

    id_usuario_actual = session.current_user['id_usuario']

    datos_abono = {
        "fk_id_venta": self.venta_actual['id_venta'],
        "fk_id_cliente": self.venta_actual['fk_id_cliente'],
        "fk_id_usuario": id_usuario_actual,
        "monto_abonado": monto_abonado,
        "metodo_pago": self.comboBox_paymentMethod.currentText(),
        "observaciones": self.textEdit_notes.toPlainText()
    }

    exito = registrar_abono_a_venta(datos_abono)

    if exito:
        QtWidgets.QMessageBox.information(self, "Éxito", "El abono ha sido registrado correctamente.")
        self.accept() # Cierra el diálogo
    else:
        QtWidgets.QMessageBox.critical(self, "Error", "No se pudo registrar el abono. Revise la consola para más detalles.")
```

limpiar_campos()

Limpia los datos del formulario cuando la búsqueda no es válida o se completa el proceso.

```
def limpiar_campos(self):
    self.venta_actual = None
    self.lineEdit_accountName.clear()
    self.doubleSpinBox_paid.setValue(0)
    self.doubleSpinBox_remaining.setValue(0)
```

e. Registrar abono.py

Esta clase gestiona la interfaz principal donde se muestran los clientes registrados y permite abrir la ventana para agregar nuevos clientes.

Constructor (__init__)

Carga la interfaz clientes.ui, conecta el botón “Nuevo Cliente” con su ventana de registro y carga los datos existentes en la tabla.

```
class ClientesWindow(QtWidgets.QMainWindow):
    def __init__(self, parent=None):
        super().__init__(parent)
        # Carga el archivo .ui directamente
        uic.loadUi("ui/clientes.ui", self)

        # Conecta el botón para abrir la ventana de registro
        self.btn_nuevo_cliente.clicked.connect(self.abrir_ventana_registro_cliente)

        self.cargar_clientes()
```

cargar_clientes()

Obtiene la lista de clientes desde la base de datos y la muestra en la tabla (QTableWidget), organizando la información por columnas (ID, nombre, correo, ciudad).

```
def cargar_clientes(self):
    clientes = listar_clientes()
    self.tblClientes.setRowCount(0)
    for row in clientes:
        rowPos = (function) tblClientes: Any count()
        self.tblClientes.insertRow(rowPosition)
        self.tblClientes.setItem(rowPosition, 0, QTableWidgetItem(str(row["id_cliente"])))
        self.tblClientes.setItem(rowPosition, 1, QTableWidgetItem(f'{row["nombre"]} {row["apellido"]}'))
        self.tblClientes.setItem(rowPosition, 2, QTableWidgetItem(row["correo"]))
        self.tblClientes.setItem(rowPosition, 3, QTableWidgetItem(row["ciudad"]))
```

abrir_ventana_registro_cliente()

Abre la ventana RegistroClienteWindow para registrar un nuevo cliente dentro del sistema.

```
def abrir_ventana_registro_cliente(self):  
    # Crea una instancia de la ventana de registro y la muestra  
    self.new_client_window = RegistroClienteWindow()  
    self.new_client_window.show()
```

f. admin_product_create.py

Esta clase define la ventana donde el administrador puede crear nuevos productos dentro del sistema. Permite ingresar, validar y guardar la información del producto en la base de datos.

guardar_producto()

Encargada de leer los datos ingresados, validarlos y enviarlos al modelo para registrarlos en la base de datos.

- Valida campos vacíos.
- Convierte precio y stock a enteros.
- Crea un diccionario con los datos del producto.
- Usa el ID del usuario en sesión para registrar el producto.

```
def guardar_producto(self):
    """
    Lee, valida y guarda los datos del nuevo producto.
    """
    # --- Paso 1: Lectura y Validación de Datos ---
    nombre = self.InputNombre.text().strip()
    categoria = self.InputCategoria.text().strip()
    marca = self.InputMarca.text().strip()
    precio_str = self.InputPrecio.text().strip()
    stock_str = self.InputStock.text().strip()

    if not all([nombre, categoria, marca, precio_str, stock_str]):
        QtWidgets.QMessageBox.warning(self, "Campos Incompletos", "Por favor, complete todos los campos obligatorios.")
        return

    try:
        precio = int(precio_str)
        stock = int(stock_str)
        if precio <= 0 or stock < 0:
            raise ValueError("El precio y el stock deben ser números positivos.")
    except ValueError:
        QtWidgets.QMessageBox.warning(self, "Datos Inválidos", "El precio y el stock deben ser números enteros válidos.")
        return

    # Recopilar todos los datos en un diccionario
    datos_producto = {
        "nombre": nombre,
        "categoria": categoria,
        "marca": marca,
        "precio": precio,
        "stock": stock,
        "descripcion": self.InputDescripcion.toPlainText().strip(),
        "estado": 1 if self.CheckActivo.isChecked() else 0
    }

    # Obtener el ID del usuario administrador desde la sesión
    if not session.current_user or 'id_usuario' not in session.current_user:
        QtWidgets.QMessageBox.critical(self, "Error de Sesión", "No se pudo identificar al usuario. Por favor, inicie sesión de nuevo.")
        return

    id_usuario_actual = session.current_user['id_usuario']

    # --- Paso 2 y 3: Inserción en la Base de Datos ---
    producto_creado = crear_producto(datos_producto, id_usuario_actual)

    if producto_creado:
        QtWidgets.QMessageBox.information(self, "Éxito", f"El producto '{nombre}' ha sido creado correctamente.")
```

Inserción en la base de datos

Si el producto se crea con éxito, muestra un mensaje de confirmación; si falla, muestra un error.

```
# --- Paso 2 y 3: Inserción en la Base de Datos ---
producto_creado = crear_producto(datos_producto, id_usuario_actual)

if producto_creado:
    QtWidgets.QMessageBox.information(self, "Éxito", f"El producto '{nombre}' ha sido creado correctamente.")
    self.accept() # Cierra el diálogo y puede ser usado para señalar éxito al padre
else:
    QtWidgets.QMessageBox.critical(self, "Error de Base de Datos", "No se pudo crear el producto. Revise la consola para más detalles.")
```

g. admin_client_list.py

Ventana principal para que el administrador (rol ADMIN) gestione todos los clientes o tenants del sistema. Permite listar, buscar, registrar y editar usuarios.

Control de Acceso

Si el usuario no tiene rol ADMIN, se muestra un mensaje de acceso denegado y se cierra la ventana automáticamente.

```
def show_access_denied(self):
    msg = QtWidgets.QMessageBox()
    msg.setIcon(QtWidgets.QMessageBox.Icon.Critical)
    msg.setText("Acceso Denegado")
    msg.setInformativeText("Solo los usuarios con rol 'ADMIN' pueden acceder a esta sección.")
    msg.setWindowTitle("Error de Permisos")
    msg.exec()
```

cargar_tenants()

Carga todos los tenants desde la base de datos y los muestra en una tabla. Cada fila incluye un botón “Ver/Editar” para abrir la ventana de edición del cliente.

```
def cargar_tenants(self):
    tenants = listar_tenants()
    self.ClientTableWidget.setRowCount(len(tenants))

    for row_position, tenant in enumerate(tenants):
        self.ClientTableWidget.setItem(row_position, 0, QtWidgets.QTableWidgetItem(str(tenant['id_usuario'])))
        self.ClientTableWidget.setItem(row_position, 1, QtWidgets.QTableWidgetItem(f'{tenant.get("nombre", "")} {tenant.get("apellido", "")}'))
        self.ClientTableWidget.setItem(row_position, 2, QtWidgets.QTableWidgetItem(tenant.get('usuario', '')))

        # Botón para editar clientes de este tenant
        btn_edit = QtWidgets.QPushButton("Ver/Editar")
        # Usamos una lambda para pasar los datos del tenant al hacer clic
        btn_edit.clicked.connect(lambda checked, t=tenant: self.abrir_edicion_cliente(t))
        self.ClientTableWidget.setCellWidget(row_position, 3, btn_edit)

    self.ClientTableWidget.resizeColumnsToContents()
```

filtrar_tenants()

Filtra los resultados de la tabla mientras se escribe en la barra de búsqueda, comparando por nombre o usuario.

```
def filtrar_tenants(self):
    texto_busqueda = self.SearchInput.text().lower()
    for i in range(self.ClientTableWidget.rowCount()):
        # Obtener el texto de las columnas relevantes
        nombre_item = self.ClientTableWidget.item(i, 1) # Columna Nombre Cliente
        usuario_item = self.ClientTableWidget.item(i, 2) # Columna Usuario (Login)

        nombre_coincide = texto_busqueda in nombre_item.text().lower() if nombre_item else False
        usuario_coincide = texto_busqueda in usuario_item.text().lower() if usuario_item else False

        self.ClientTableWidget.setRowHidden(i, not (nombre_coincide or usuario_coincide))
```

abrir_edicion_cliente()

Abre la ventana para editar la información de un cliente específico.

```
def abrir_edicion_cliente(self, tenant_data):
    """
    Abre la ventana de edición para un cliente específico de un tenant.
    Aquí se debería abrir una lista de clientes de ESE tenant, y de ahí
    pasar a la ventana de edición.

    Por simplicidad y para seguir el flujo, abriremos la ventana de edición
    directamente, asumiendo que se quiere editar el cliente con ID 1 de ese tenant.
    En una implementación real, aquí iría la ventana de lista de clientes del tenant.
    """
    # Obtenemos el ID del usuario (que es el cliente en este contexto) de los datos del tenant
    usuario_a_editar_id = tenant_data['id_usuario']

    self.edit_window = AdminClientEditWindow(cliente_id=usuario_a_editar_id)
    self.edit_window.show()
```

abrir_registro_usuario()

Abre la ventana para registrar un nuevo usuario o tenant.

```
def abrir_registro_usuario(self):
    self.registro_window = RegistroUsuarioWindow()
    self.registro_window.show()
```

cerrar_sesion()

Cierra la sesión actual y redirige de nuevo a la ventana de inicio de sesión (LoginWindow).

```
def cerrar_sesion(self):  
    # Importación local para romper el ciclo de importación  
    from main import LoginWindow  
    self.login_window = LoginWindow()  
    self.login_window.show()  
    self.close()
```

h. admin_client_edit.py

Ventana de administración que permite editar la información de un usuario o cliente. Incluye carga automática de datos, actualización de registros y restablecimiento de contraseña.

setup_connections()

Conecta los botones con sus respectivas funciones (guardar_cambios, restablecer_contrasena, close).

```
def setup_connections(self):  
    self.SaveClientButton.clicked.connect(self.guardar_cambios)  
    self.ResetPasswordButton.clicked.connect(self.restablecer_contrasena)  
    self.CancelButton.clicked.connect(self.close)
```

cargar_datos_cliente()

Carga información desde dos tablas:

- **usuario** → nombre, apellido, usuario.
- **cliente** → correo, dirección, ciudad, departamento, estado.
Si el cliente no existe, muestra campos vacíos.

```
def cargar_datos_cliente(self):
    """
    Implementa el flujo de carga de datos desde ambas bases de datos.
    """
    # 1. Cargar datos del USUARIO (login) desde la tabla 'usuario'
    usuario_data = obtener_usuario_por_id(self.user_id)

    if not usuario_data:
        QtWidgets.QMessageBox.critical(self, "Error", f"No se encontró el usuario con ID {self.user_id} en la base de datos.")
        self.close()
        return

    # Llenar campos de la tabla 'usuario'
    self.InputNombre.setText(usuario_data.get('nombre', ''))
    self.InputApellido.setText(usuario_data.get('apellido', ''))
    self.InputUsername.setText(usuario_data.get('usuario', ''))
    self.TitleLabel.setText(f"Editar Usuario: {usuario_data.get('nombre', '')} {usuario_data.get('apellido', '')}")

    # 2. Cargar datos del CLIENTE (perfil) desde la tabla 'cliente'
    cliente_data = obtener_cliente_por_usuario_id(self.user_id)
    if cliente_data:
        self.cliente_profile_id = cliente_data.get('id_cliente')
        self.InputNumDoc.setText(cliente_data.get('num_doc', ''))
        self.InputCorreo.setText(cliente_data.get('correo', ''))
        self.InputDireccion.setText(cliente_data.get('direccion', ''))
        self.InputCiudad.setText(cliente_data.get('ciudad', ''))
        self.InputDepartamento.setText(cliente_data.get('departamento', ''))
        self.CheckActivo.setChecked(bool(cliente_data.get('activo', True)))
    else:
        # Si no hay perfil de cliente, los campos se quedan vacíos
        self.InputCorreo.setPlaceholderText("Aún no se ha creado un perfil de cliente")
```

guardar_cambios()

Guarda o actualiza la información del cliente y del usuario:

- Actualiza datos personales y de contacto.
- Modifica también el nombre o usuario de inicio de sesión.
Muestra mensaje de éxito o error según el resultado.

```
def guardar_cambios(self):
    """
    Implementa el flujo de guardado en ambas bases de datos.
    """
    # 1. Guardar/Actualizar datos del perfil en la tabla 'cliente'
    datos_perfil_cliente = {
        'nombre': self.InputNombre.text(),
        'apellido': self.InputApellido.text(),
        'num_doc': self.InputNumDoc.text(),
        'correo': self.InputCorreo.text(),
        'direccion': self.InputDireccion.text(),
        'ciudad': self.InputCiudad.text(),
        'departamento': self.InputDepartamento.text(),
        'activo': self.CheckActivo.isChecked(),
        'fk_id_usuario': self.user_id # Vínculo clave
    }
    perfil_actualizado = actualizar_o_crear_cliente(datos_perfil_cliente)

    # 2. Actualizar datos de login en la tabla 'usuario'
    usuario_actualizado = actualizar_username_y_clave(
        id_usuario=self.user_id,
        nombre=self.InputNombre.text(),
        apellido=self.InputApellido.text(),
        usuario=self.InputUsername.text()
    )

    if perfil_actualizado and usuario_actualizado:
        QtWidgets.QMessageBox.information(self, "Éxito", "Los datos del cliente y/o usuario han sido actualizados.")
        self.accept() # Cierra la ventana indicando éxito
    else:
        QtWidgets.QMessageBox.warning(self, "Error", "No se pudieron guardar todos los cambios. Revise la consola para más detalles.")
```

restablecer_contraseña()

Permite cambiar la contraseña del usuario, verificando coincidencia de los campos y actualizando en la base de datos.

```
def restablecer_contraseña(self):
    """
    Implementa el flujo de cambio de contraseña en la DB Central.
    """
    nueva_clave = self.InputNewPassword.text()
    confirmar_clave = self.InputConfirmPassword.text()

    if not nueva_clave:
        QtWidgets.QMessageBox.warning(self, "Campo Vacío", "Por favor, ingrese la nueva contraseña.")
        return

    if nueva_clave != confirmar_clave:
        QtWidgets.QMessageBox.warning(self, "Error", "Las contraseñas no coinciden.")
        return

    # Llamar a la función del modelo que hashea y actualiza la clave
    if actualizar_username_y_clave(id_usuario=self.user_id, clave=nueva_clave):
        QtWidgets.QMessageBox.information(self, "Éxito", "La contraseña ha sido restablecida correctamente.")
        self.InputNewPassword.clear()
        self.InputConfirmPassword.clear()
    else:
        QtWidgets.QMessageBox.critical(self, "Error", "No se pudo restablecer la contraseña.")
```